

Теория формальных языков
Лабораторная работа №4

Вариант №26

Чайкин Семён

1. Задание

Дано расширенное регулярное выражение

$$\wedge(a|b\backslash 1b)^*c((?:a|b)^*)((?:a|b)^*)((?:a|b)^*)c(?:= \backslash 1\$)(?:= \backslash 2\backslash 3\$)\backslash 3\backslash 4\$$$

Необходимо

- Проанализировать на КС-свойство/регулярность.
- Построить “наивный” парсер, используя рекурсивный разбор с возвратом.
- Построить оптимизированный парсер, оценить его сложность

2. Анализ на КС-свойство

Для начала упростим расширенное регулярное выражение. Заметим, что из условия $(?:= \backslash 1\$)(?:= \backslash 2\backslash 3\$)\backslash 3\backslash 4\$$ следует, что $\backslash 2\backslash 3 = \backslash 1$, $\backslash 3\backslash 4 = \backslash 1$.

Заметим, также, что из $\backslash 2\backslash 3 = \backslash 3\backslash 4 = \backslash 1$ и $\backslash 1 = b^nab^n$ следует, что внутренний блок между c можно представить в виде b^nab^n или $b^nab^mab^n$, где $n \geq 0$, $n \leq m \leq 2 \cdot n$.

Докажем, что язык не контекстно-свободный. Для этого пересечём его с языком $R = a(b^+ab^+)^*acacac$. Так как внутри скобок b^+ , то при итерации выражение $(a|b\backslash 1b)$ раскрывается только по второй альтернативе. Пусть w_i — слово после i -го раскрытия по второй альтернативе. Очевидно, что $w_i = b^iab^i$. Пусть $w_0 = a$. Тогда язык

$$L_1 = L \cap R = \{w_0w_1\dots w_nacacac \mid n \geq 0\}$$

Рассмотрим $\Psi_\Sigma(L_1)$, $\Sigma = \{a, b, c\}$. Предположим, что было n -итераций ($n \geq 0$). Тогда $|w|_a = 1 + n + 3 = n + 4$, $|w|_c = 2$, $|w|_b = \underbrace{2 + 4 + \dots + 2 \cdot n}_{n \text{ элементов}} = n(n + 1)$.

Получаем

$$\Psi_\Sigma(L_1) = \{(n + 4, n \cdot (n + 1), 2) \mid n \geq 0\}$$

Можно доказать, что $\Psi_\Sigma(L_1)$ не является полулинейным, значит, язык L_1 КС, а значит и исходный язык L не КС.

3. Наивный парсер

Находится в структуре `NaiveParser`.

4. Оптимизированный парсер

Главная идея по оптимизации была описана ранее, поэтому приведём только анализ сложности алгоритма.

Предположим, что получаемая строка записана в виде $LcMcR$.

Подвыражение $(a|b\backslash 1b)^*$ (блок L) достаточно просто можно проверить за $O(|L|)$ времени (сначала считываем a , после этого в зависимости от первой буквы после

очередного блока решаем — скидываем счётчик глубины h до 0, если прочитали a , или читаем блок $b^h a b^h$). После этого достаточно знать h для последнего блока.

Проверка центрального блока между s проверяется крайне просто за $O(|M|)$, так как достаточно просто проверить, что строка имеет вид $b^n a b^m a b^n$, где $n = h$, $0 \leq m \leq 2 \cdot n$.

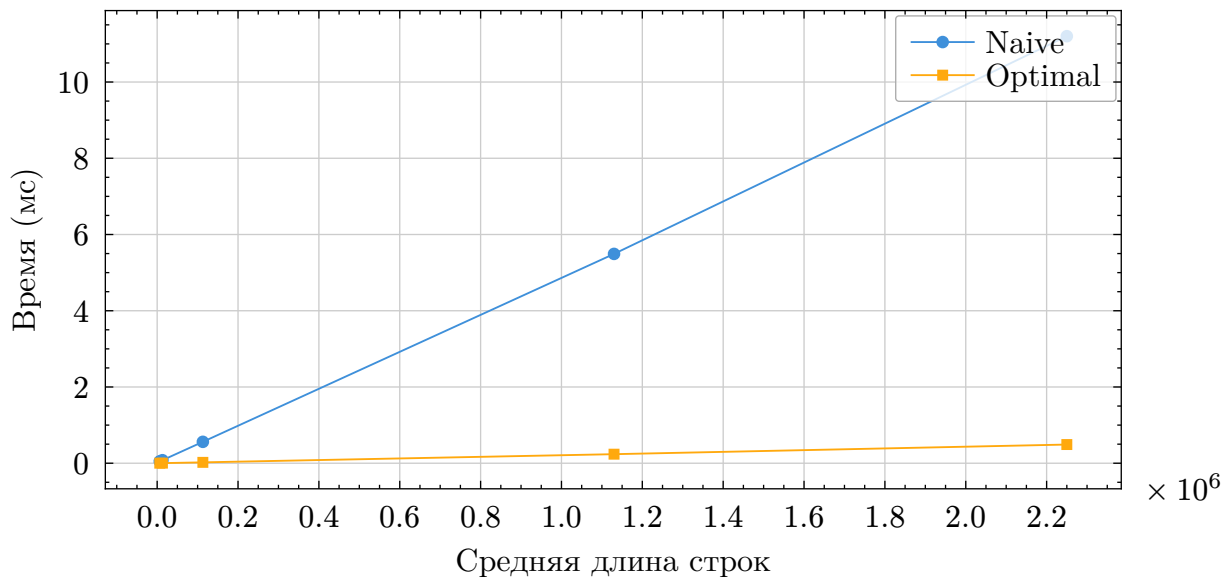
Правый блок R просто совпадает с $\setminus 1$, а значит просто проверяем, что имеет вид $b^h a b^h$. Это тоже за $O(|R|)$.

Сложность алгоритма — $O(|\omega|)$.

Находится в структуре `OptimizedParser`.

5. Фазз-тестирование и сравнение скорости работы

5.1. Слова из языка



5.2. Слова не из языка

